

Rapport de Projet : Spatially Balanced Latin Square (SBLs)

Carré Matthias

Janvier 2026

Table des matières

1	Introduction	3
2	Modélisation en Programmation par Contraintes	3
2.1	Variables de décision et Domaines	3
2.2	Contraintes utilisées	3
2.3	Algorithmes de filtrage	4
3	Stratégies de Résolution	4
3.1	Cassage de symétries	4
3.2	Heuristiques de choix de variables et de valeurs	4
4	Expérimentations et Résultats	4
4.1	Environnement de test	4
4.2	Analyse des performances	4
5	Conclusion	5
6	Annexe	5

1 Introduction

Le problème des Carrés Latins Spatialement Équilibrés (*Spatially Balanced Latin Square* - SBLS) trouve une application directe et concrète dans le domaine de l'expérimentation agronomique. L'enjeu est de tester n engrais différents (modélisés par des couleurs ou des entiers) sur un ensemble de parcelles disposées en grille $n \times n$.

L'utilisation d'un simple carré latin (où chaque engrais apparaît une seule fois par ligne et par colonne) ne suffit pas toujours à garantir l'absence de biais :

- **Biais environnementaux** : La proximité d'une lisière de forêt ou d'une route peut influencer les résultats d'un engrais s'il est mal réparti.
- **Interactions de voisinage** : Si deux engrais spécifiques se retrouvent systématiquement côte à côte, des échanges chimiques ou biologiques entre parcelles peuvent fausser les mesures.

L'objectif des SBLS est d'équilibrer ces voisinages. Mathématiquement, on impose que pour chaque paire de couleurs (a, b) , la somme des distances entre ces couleurs dans le carré soit constante. Ce rapport présente la modélisation et la résolution de ce problème via la **Programmation par Contraintes**.

2 Modélisation en Programmation par Contraintes

2.1 Variables de décision et Domaines

Pour modéliser ce problème, nous utilisons une structure de données articulée autour de trois axes :

1. **Matrice principale** $X[n][n]$: Chaque cellule $X_{i,j}$ possède un domaine $D = \{1, \dots, n\}$ représentant l'engrais affecté.
2. **Matrices de position** : Pour faciliter le calcul des distances, nous utilisons des variables de position (canalisées avec la matrice principale) permettant de retrouver rapidement l'indice de ligne ou de colonne d'un engrais donné.
3. **Variable globale** K : Une variable (ou un ensemble de variables) servant à représenter la somme des distances, permettant de vérifier l'équilibre spatial entre toutes les paires d'engrais.

2.2 Contraintes utilisées

Le modèle repose sur trois piliers de contraintes :

- **Structure de Carré Latin** : Utilisation de la contrainte globale `allDifferent` sur chaque ligne et chaque colonne de la matrice X .
- **Calcul des distances** : Pour chaque paire de valeurs $(v_1, v_2) \in \{1, \dots, n\}^2$, on calcule la distance $|pos(v_1) - pos(v_2)|$.
- **Équilibre spatial** : La somme de ces distances pour chaque paire doit être égale à une constante K .

2.3 Algorithmes de filtrage

Pour les contraintes **allDifferent**, nous privilégions l’algorithme de **filtrage basé sur l’Arc-Consistance (AC)** plutôt que le *Forward Checking*. Bien que plus coûteux en temps de calcul par nœud, il permet une réduction drastique du domaine et limite les retours sur trace (*backtracks*) sur des instances de taille moyenne.

3 Stratégies de Résolution

Initialement, la contrainte d’équilibre spatial n’était appliquée que sur les lignes. Cependant, pour respecter la logique agronomique bidimensionnelle, le modèle a été étendu pour inclure les distances en colonnes, rendant le problème beaucoup plus contraint.

3.1 Cassage de symétries

Le problème du carré latin est hautement symétrique (permutations de lignes, de colonnes ou de valeurs). Pour réduire l’espace de recherche, nous avons fixé les valeurs de la **première ligne** (par exemple : $1, 2, \dots, n$). Cette technique simple élimine $n!$ symétries triviales.

3.2 Heuristiques de choix de variables et de valeurs

Nous utilisons l’heuristique **DomOverWDeg** pour le choix des variables. Elle privilégie les variables ayant un petit domaine et impliquées dans des contraintes ayant souvent échoué, ce qui est particulièrement efficace pour les problèmes de type "grille" très contraints.

4 Expérimentations et Résultats

4.1 Environnement de test

Les tests ont été effectués sur la configuration suivante :

- **Processeur** : Intel Core i7-8700
- **Mémoire RAM** : 32 Go
- **Outils** : Choco Solver 4.10.14, Java 21.

4.2 Analyse des performances

Les tableaux de résultats mettent en évidence l’impact des optimisations :

Ordre n	Temps (s)	Nœuds	Backtracks	#Solution
5 (Modèle simple)	2,48	46 053	80 587	5760
5 (Symétrie cassée)	0,11	583	1 071	48

TABLE 1 – Comparaison de l’impact du cassage de symétrie (Extrait)

Ordre n	Temps (s)	Nœuds	Backtracks	#Solution
6 (lignes seulement)	3,01	43 072	82 305	1920
6 (lignes et colonnes)	6,44	61 021	121 787	128

TABLE 2 – Comparaison de l’impact d’ajout de contrainte (Extrait)

L’observation de la table 1 montre que l’ajout de contraintes de cassage de symétrie réduit drastiquement l’espace de recherche : pour $n = 5$, on passe de 46 000 nœuds à seulement 583. Cela permet de résoudre l’instance $n = 6$ en seulement 3 secondes, alors qu’elle était incalculable auparavant.

Cependant, la table 2 révèle un phénomène intéressant : l’ajout des contraintes sur les colonnes augmente le nombre de nœuds (de 43 000 à 61 000 pour $n = 6$). Cela s’explique par le fait que l’on ajoute une dimension de complexité au problème (le problème devient "plus dur" à satisfaire), ce qui ralentit la résolution malgré un élagage théorique plus fort.

Voir plus de détails dans les Tables 3,4,5

5 Conclusion

Ce projet a permis de démontrer l’efficacité de la programmation par contraintes pour résoudre des problèmes de conception expérimentale complexes. L’utilisation de **Choco Solver** a mis en évidence l’importance cruciale du **cassage de symétries**, qui reste le levier le plus puissant pour passer à l’échelle supérieure.

Le modèle atteint ses limites pour $n = 7$ en raison de l’explosion combinatoire des calculs de distances entre paires. Pour aller plus loin, il serait intéressant d’explorer des approches de recherche locale ou des contraintes globales spécifiques au calcul de distance spatiale.

6 Annexe

Ordre n	Temps (s)	Nœuds	Backtracks	#Solution
2	0,04	3	3	2
3	0,02	31	39	12
4	0,1	687	1375	0
5	2,48	46 053	80 587	5760
6	x	x	x	x
7	x	x	x	x

TABLE 3 – Résultats pour le problème sans suppression des symétrie et seulement la contrainte sur les lignes

Ordre n	Temps (s)	Nœuds	Backtracks	#Solution
2	0,03	1	1	1
3	0,01	3	3	2
4	0,18	23	47	0
5	0,11	583	1 071	48
6	3,01	43 072	82 305	1920
7	x	x	x	x

TABLE 4 – Résultats pour le problème avec suppression des symétrie et seulement la contrainte sur les lignes

Ordre n	Temps (s)	Nœuds	Backtracks	#Solution
2	0,04	1	1	1
3	0,01	3	3	2
4	0,18	41	83	0
5	0,12	515	999	16
6	6,44	61 021	121 787	128
7	x	x	x	x

TABLE 5 – Résultats pour le problème avec suppression des symétrie et contrainte sur ligne et colonnes